

```

import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
import re

conversations = [
    ("hello", "hi how are you"),
    ("how are you", "i am fine"),
    ("what is your name", "i am a chatbot"),
    ("good morning", "good morning to you"),
    ("bye", "goodbye have a nice day")
]

def clean_text(text):
    return re.sub(r"^[a-zA-Z0-9\s]", "", text.lower())

input_texts = [clean_text(pair[0]) for pair in conversations]
target_texts = [clean_text(pair[1]) for pair in conversations]

all_words = " ".join(input_texts + target_texts).split()
vocab = ["<PAD>", "<SOS>", "<EOS>"] + sorted(list(set(all_words)))
word2idx = {w: i for i, w in enumerate(vocab)}
idx2word = {i: w for i, w in enumerate(vocab)}
vocab_size = len(vocab)

def tokenize(sentence):
    return [word2idx[w] for w in sentence.split()]

```

```

class Encoder(nn.Module):
    def __init__(self, vocab_size, embed_size, hidden_size):
        super(Encoder, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embed_size)
        self.lstm = nn.LSTM(embed_size, hidden_size, batch_first=True)

    def forward(self, x):
        embedded = self.embedding(x)
        outputs, (hidden, cell) = self.lstm(embedded)
        return outputs, hidden, cell

class Attention(nn.Module):
    def __init__(self, hidden_size):
        super(Attention, self).__init__()
        self.attn = nn.Linear(hidden_size * 2, hidden_size)
        self.v = nn.Linear(hidden_size, 1, bias=False)

    def forward(self, hidden, encoder_outputs):
        seq_len = encoder_outputs.shape[1]
        hidden_repeated = hidden[-1].unsqueeze(1).repeat(1, seq_len, 1)
        energy = torch.tanh(self.attn(torch.cat((hidden_repeated, encoder_outputs), dim=2)))
        attention_scores = self.v(energy).squeeze(2)
        return F.softmax(attention_scores, dim=1)

class DecoderWithAttention(nn.Module):
    def __init__(self, vocab_size, embed_size, hidden_size):
        super(DecoderWithAttention, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embed_size)
        self.lstm = nn.LSTM(embed_size + hidden_size, hidden_size, batch_first=True)
        self.fc = nn.Linear(hidden_size, vocab_size)
        self.attention = Attention(hidden_size)

    def forward(self, x, hidden, cell, encoder_outputs):
        x = x.unsqueeze(1)
        embedded = self.embedding(x)
        attn_weights = self.attention(hidden, encoder_outputs)
        context = torch.bmm(attn_weights.unsqueeze(1), encoder_outputs)
        rnn_input = torch.cat((embedded, context), dim=2)
        output, (hidden, cell) = self.lstm(rnn_input, (hidden, cell))
        prediction = self.fc(output.squeeze(1))
        return prediction, hidden, cell, attn_weights

class Seq2SeqAttention(nn.Module):
    def __init__(self, encoder, decoder):
        super(Seq2SeqAttention, self).__init__()
        self.encoder = encoder
        self.decoder = decoder

    def forward(self, source, target, teacher_forcing_ratio=0.5):
        batch_size = source.shape[0]

```

```

target_len = target.shape[1]
vocab_size = self.decoder.fc.out_features
outputs = torch.zeros(batch_size, target_len, vocab_size).to(source.device)

encoder_outputs, hidden, cell = self.encoder(source)
x = target[:, 0]

for t in range(1, target_len):
    output, hidden, cell, _ = self.decoder(x, hidden, cell, encoder_outputs)
    outputs[:, t] = output
    top1 = output.argmax(1)
    x = target[:, t] if torch.rand(1).item() < teacher_forcing_ratio else top1

return outputs

```

```

embed_size = 64
hidden_size = 128

encoder = Encoder(vocab_size, embed_size, hidden_size)
decoder = DecoderWithAttention(vocab_size, embed_size, hidden_size)
model = Seq2SeqAttention(encoder, decoder)

criterion = nn.CrossEntropyLoss(ignore_index=word2idx["<PAD>"])
optimizer = optim.Adam(model.parameters(), lr=0.01)

max_len = 10

def pad_sequence(seq, max_len):
    return seq + [word2idx["<PAD>"]] * (max_len - len(seq))

src_data = torch.tensor([pad_sequence(tokenize(text), max_len) for text in input_texts], dtype=torch.long)
trg_data = torch.tensor([pad_sequence([word2idx["<SOS>"]] + tokenize(text) + [word2idx["<EOS>"]], max_len) for text in target_texts], dtype=torch.long)

epochs = 150
for epoch in range(epochs):
    optimizer.zero_grad()
    output = model(src_data, trg_data)
    output = output[:, 1:].reshape(-1, vocab_size)
    target = trg_data[:, 1:].reshape(-1)

    loss = criterion(output, target)
    loss.backward()
    optimizer.step()

```

```

def evaluate_and_show_attention(model, sentence):
    model.eval()
    with torch.no_grad():
        tokens = tokenize(clean_text(sentence))
        src_tensor = torch.tensor(tokens, dtype=torch.long).unsqueeze(0)
        encoder_outputs, hidden, cell = model.encoder(src_tensor)

        x = torch.tensor([word2idx["<SOS>"]], dtype=torch.long)
        decoded_words = []
        attentions = []

        for _ in range(10):
            output, hidden, cell, attn_weights = model.decoder(x, hidden, cell, encoder_outputs)
            attentions.append(attn_weights.squeeze().tolist())
            top1 = output.argmax(1).item()

            if top1 == word2idx["<EOS>"]:
                break

            decoded_words.append(id2word[top1])
            x = torch.tensor([top1], dtype=torch.long)

        return " ".join(decoded_words), attentions

test_input = "how are you"
response, attn = evaluate_and_show_attention(model, test_input)

print(f"Input: {test_input}")
print(f"Output: {response}")
print(f"Attention Weights: {attn}")

```

```

Input: how are you
Output:
Attention Weights: [[0.022787759080529213, 0.42385727167129517, 0.5533549785614014]]

```

